AWS VPC PEERING

EWB Cloud Labs — Student Project Report

Student	Siva Durga Prasad Kavicharla
Project	Private IPv4 connectivity between two Amazon VPCs
AWS Region	Asia Pacific (Mumbai) — ap-south-1
VPC peering ID	pcx-03fca277b8a48c690
Evidence activity date	23 October 2026
Batch	1

Project objective

Build private IPv4 communication between two non-overlapping Amazon VPCs using VPC peering. Keep both application servers on private IP addresses, use a public bastion as the SSH administration jump point, and validate connectivity with bidirectional ping tests and a controlled ICMP rule change.

1. Executive Summary

The project connects **ewb-vpc-a** (10.10.0.0/16) and **ewb-vpc-b** (10.20.0.0/16) in the Mumbai Region through the active VPC peering connection **pcx-03fca277b8a48c690**. The VPC CIDR blocks do not overlap, so they can be connected by peering routes.

The supplied terminal evidence shows four successful ICMP replies in each direction: App-A (10.10.2.10) to App-B (10.20.1.10), and App-B to App-A, with 0% packet loss. Additional supplied evidence shows SSH access to App-B and a controlled ICMP failure followed by recovery. The AWS console evidence confirms the active peering connection, VPC CIDRs, the public route table's peering route, and the three EC2 instances running with the expected private addresses.

The report distinguishes direct screenshot evidence from details described in the project notes. Screenshots for security-group rules and both private route tables were not supplied.

2. Architecture

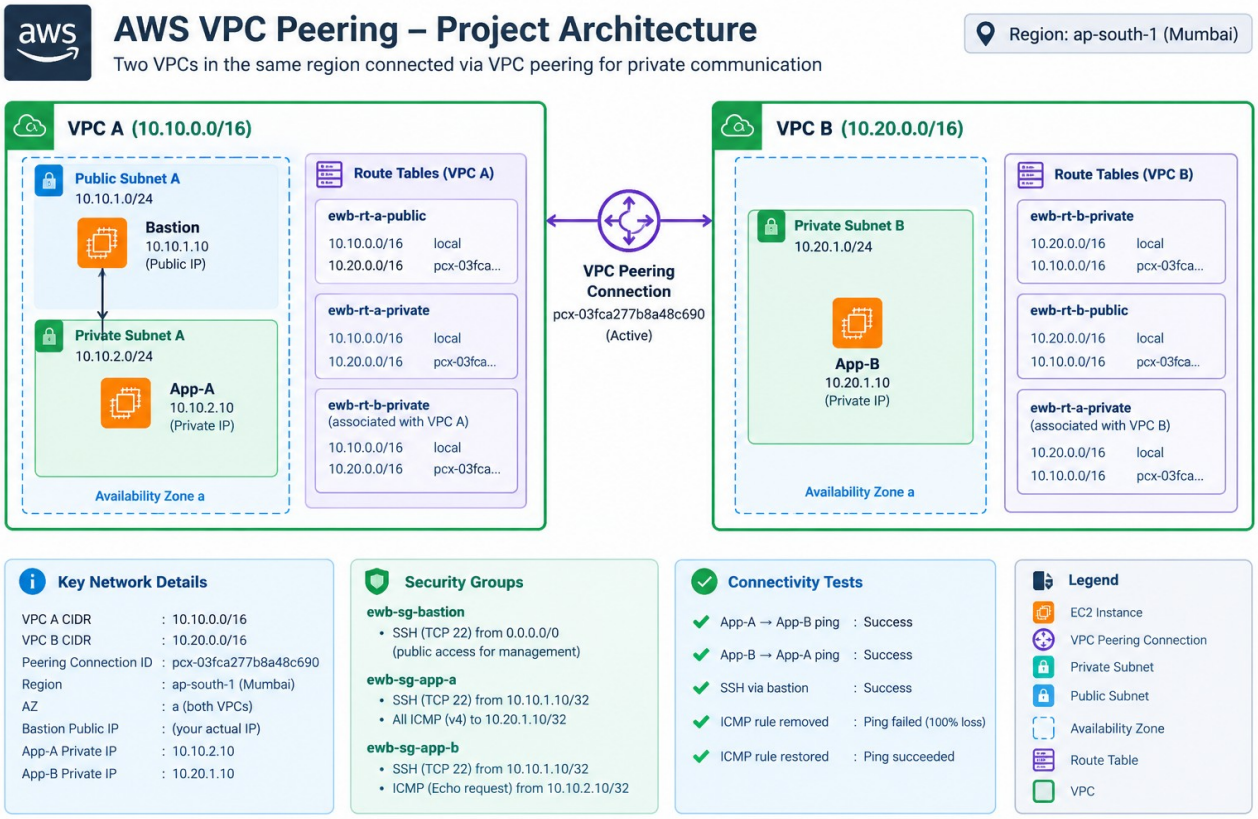


Figure 1. Supplied project architecture diagram. It depicts both VPCs, subnet ranges, bastion and application instances, route tables, peering connection, security-group intent, and reported connectivity checks.

Administration path: administrator → bastion public IPv4 → private App-A or App-B over SSH. The bastion subnet route table needs a route to VPC B for SSH ProxyJump to App-B, and VPC B needs a return route to VPC A.

Application path: App-A → VPC A private route table → VPC peering → VPC B private route table → App-B. Replies follow the reverse route. VPC peering is not transitive and does not give VPC B internet access through VPC A.

3. Resource Inventory and Address Plan

Resource	Name	CIDR / private IP	Notes / evidence
VPC A	ewb-vpc-a	10.10.0.0/16	VPC console screenshot supplied
Public subnet A	ewb-a-public	10.10.1.0/24	Subnet ID not supplied
Private subnet A	ewb-a-private	10.10.2.0/24	Subnet ID not supplied
VPC B	ewb-vpc-b	10.20.0.0/16	VPC console screenshot supplied
Private subnet B	ewb-b-private	10.20.1.0/24	Subnet ID not supplied
Bastion	ewb-bastion	10.10.1.10	Public IPv4 13.127.110.57 in capture; verify if still current
App-A	ewb-app-a	10.10.2.10	No public IPv4 displayed; Ubuntu
App-B	ewb-app-b	10.20.1.10	No public IPv4 displayed; Ubuntu
VPC peering	ewb-a-b-peer	N/A	pcx-03fca277b8a48c690; Active
Internet gateway	ewb-igw-a	N/A	ID not supplied
Route tables	ewb-rt-a-public; ewb-rt-a-private; ewb-rt-b-private	N/A	Only public route table screenshot supplied

Operating-system deviation

The workbook recommended Amazon Linux 2023, but this implementation used Ubuntu. The terminal prompts show the Ubuntu login account **ubuntu**. This difference is recorded as an implementation deviation; the supplied networking tests still demonstrate private reachability.

4. Implementation Procedure

Step 1 — Create the VPCs

Create ewb-vpc-a with 10.10.0.0/16 and ewb-vpc-b with 10.20.0.0/16. Confirm the CIDR ranges do not overlap.

Step 2 — Create the subnets

Create public subnet A (10.10.1.0/24), private subnet A (10.10.2.0/24), and private subnet B (10.20.1.0/24).

Step 3 — Launch instances

Launch ewb-bastion (10.10.1.10), ewb-app-a (10.10.2.10), and ewb-app-b (10.20.1.10). App-A and App-B should remain without public IPv4 addresses.

Step 4 — Establish peering

Create the peering connection between the VPCs and confirm it is Active. The supplied console screenshot confirms ID pcx-03fca277b8a48c690.

Step 5 — Configure routing

Add the opposite VPC CIDR route to the peering connection in the relevant route tables on both sides. Add the VPC B route to the bastion subnet route table if using SSH ProxyJump to App-B.

Step 6 — Configure security groups

Restrict bastion SSH to the administrator public IPv4 /32; allow application SSH from the bastion; allow ICMP Echo Request between the two application IPs only for the intended test.

Step 7 — Validate and document

Run bidirectional ping tests, verify SSH through the bastion, remove the App-B ICMP rule to test failure, then restore it and verify recovery.

5. Routing Configuration

Route table	Destination	Target	Purpose
ewb-rt-a-public	10.10.0.0/16	local	Existing VPC A route
ewb-rt-a-public	0.0.0.0/0	Internet gateway	Bastion subnet internet route
ewb-rt-a-public	10.20.0.0/16	pcx-03fca277b8a48c690	Bastion → App-B SSH path
ewb-rt-a-private	10.10.0.0/16	local	Existing VPC A route
ewb-rt-a-private	10.20.0.0/16	pcx-03fca277b8a48c690	App-A → App-B
ewb-rt-b-private	10.20.0.0/16	local	Existing VPC B route
ewb-rt-b-private	10.10.0.0/16	pcx-03fca277b8a48c690	Return path to VPC A

The uploaded public route-table screenshot shows local 10.10.0.0/16, peering route 10.20.0.0/16→ pcx-03fca277b8a48c690, and default route 0.0.0.0/0 to an internet gateway. The private route entries above reflect the project configuration supplied by the user, but their own console screenshots were not included.

Why return routes matter

Routing must be configured in both VPCs. A request can reach the peer while the response fails if the destination side lacks a route back to the source CIDR. The bastion subnet also requires the VPC B route when the bastion itself initiates an SSH connection to App-B.

6. Security Groups and Access Control

Security group	Inbound rule	Source	Purpose
ewb-sg-bastion	SSH TCP 22	Administrator public IPv4 /32	SSH entry point; verify current source
ewb-sg-app-a	SSH TCP 22	10.10.1.10/32	SSH from bastion
ewb-sg-app-a	ICMPv4 Echo Request	10.20.1.10/32	Ping from App-B
ewb-sg-app-b	SSH TCP 22	10.10.1.10/32	SSH from bastion
ewb-sg-app-b	ICMPv4 Echo Request	10.10.2.10/32	Ping from App-A

These rules are based on the supplied project details and architecture diagram; dedicated security-group console screenshots were not supplied. Verify the final live rules before submission. All three groups were described as retaining default allow-all outbound rules; the project notes also indicate default network ACL behavior.

Security recommendations

• Do not expose SSH to 0.0.0.0/0. Restrict it to the administrator's current public IPv4 /32.

• Keep both application servers private and allow SSH only from the bastion private IP.

• Keep ICMP rules as narrow as possible and remove them if they are not required after testing.

• Never include private keys, passwords, access tokens, or other secrets in the report.

7. Validation and Test Results

Test	Command / action	Observed result	Evidence status
Forward ping	App-A: ping -c 4 10.20.1.10	4 received; 0% loss	Screenshot supplied
Reverse ping	App-B: ping -c 4 10.10.2.10	4 received; 0% loss	Screenshot supplied
SSH to App-B	SSH from local terminal using ewb-app-b alias; session shows Ubuntu host	Connected; hostname ip-10-20-1-10	Screenshot supplied
Controlled failure	Ping from App-A after App-B ICMP rule removed	4 transmitted, 0 received; 100% loss	Screenshot supplied
Recovery	Restore App-B ICMP rule; ping from App-A	4 received; 0% loss	Screenshot supplied
Initial baseline before routes	Initial App-A → App-B ping	Reported baseline failure	Initial baseline-failure screenshot not captured

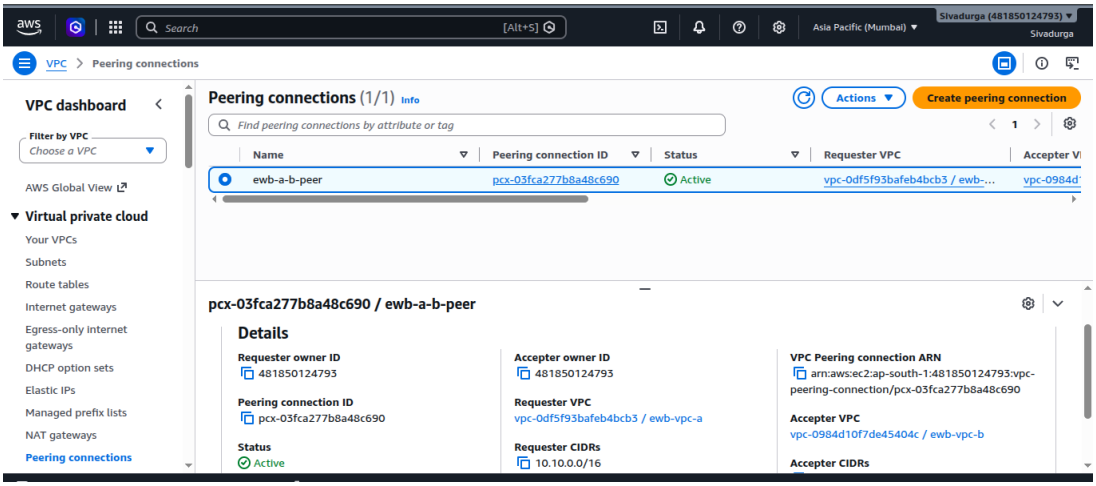
Interpretation

The supplied terminal evidence directly supports successful bidirectional ICMP communication after configuration. The controlled-failure screenshot shows 100% packet loss when the App-B ICMP rule was removed; the recovery screenshot shows four replies and 0% loss after the rule was restored. The separate initial baseline-failure screenshot was not captured, so that earlier result is disclosed as reported rather than visually verified.

The SSH screenshot shows a terminal session on App-B with private address 10.20.1.10 and hostname ip-10-20-1-10. The project notes describe the access path as via the bastion; this is consistent with the private-only address design.

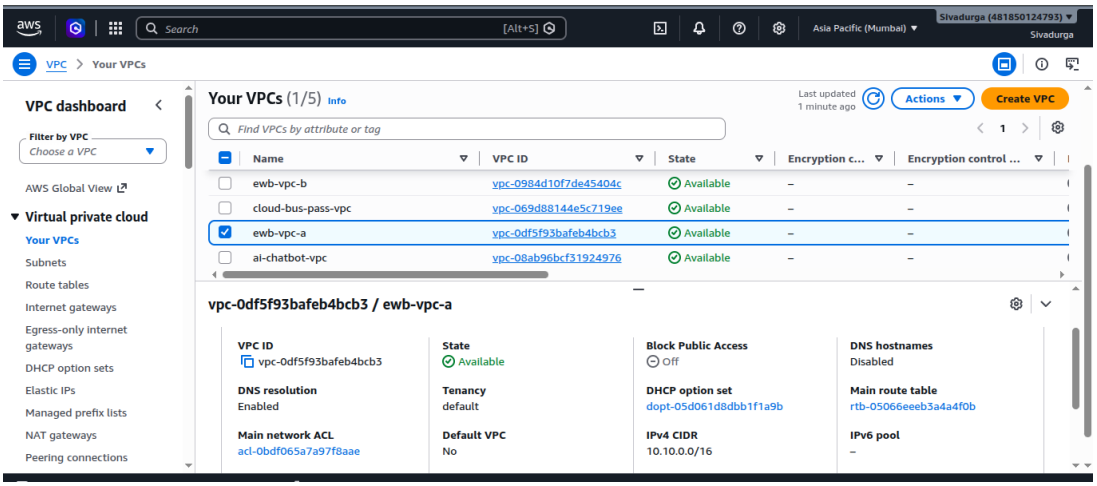
8. Evidence Gallery — AWS Console

Evidence E1 — Active VPC peering



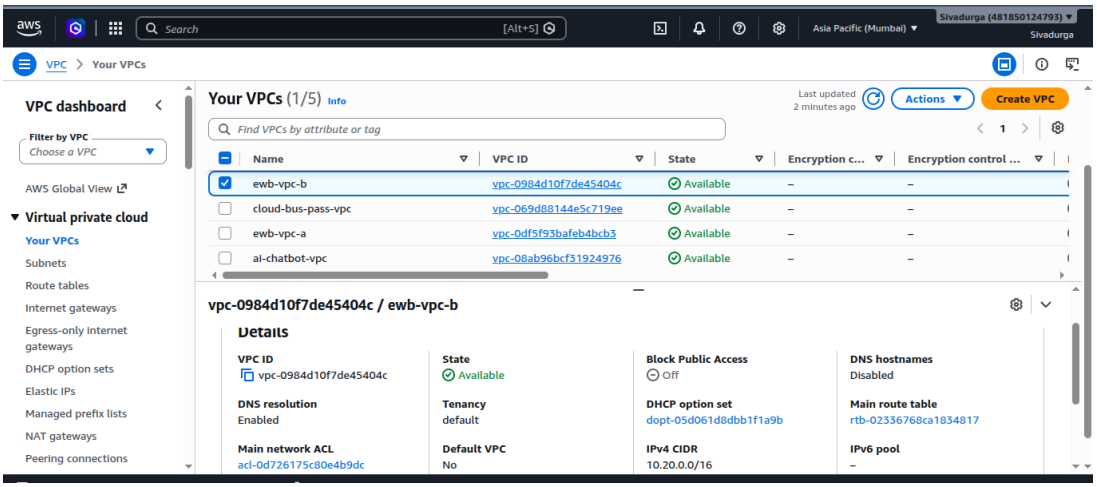
The console displays ewb-a-b-peer, ID pcx-03fca277b8a48c690, in Active state.

Evidence E2 — VPC A CIDR



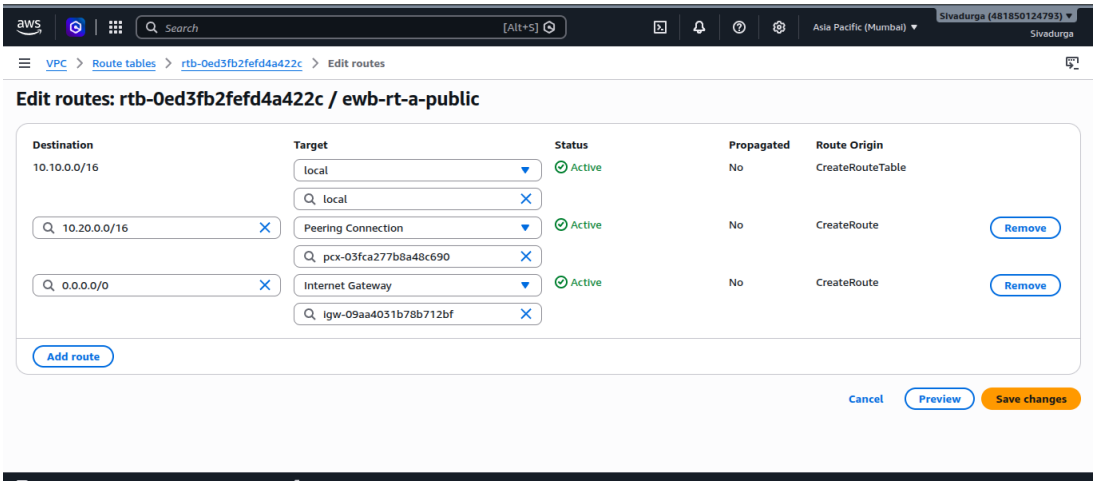
The console displays ewb-vpc-a with CIDR 10.10.0.0/16.

Evidence E3 — VPC B CIDR



The console displays ewb-vpc-b with CIDR 10.20.0.0/16.

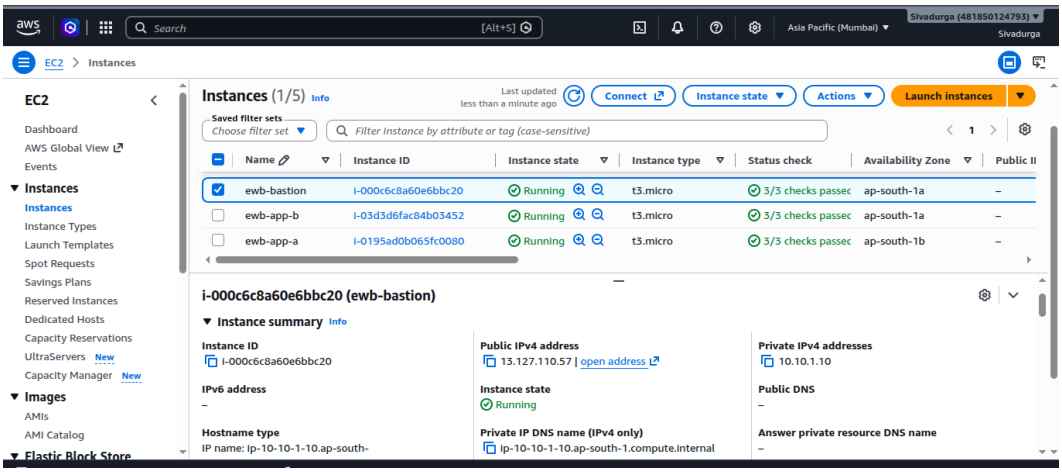
Evidence E4 — Public route table



The visible routes include local VPC A, 10.20.0.0/16 to the peering connection, and 0.0.0.0/0 to an internet gateway.

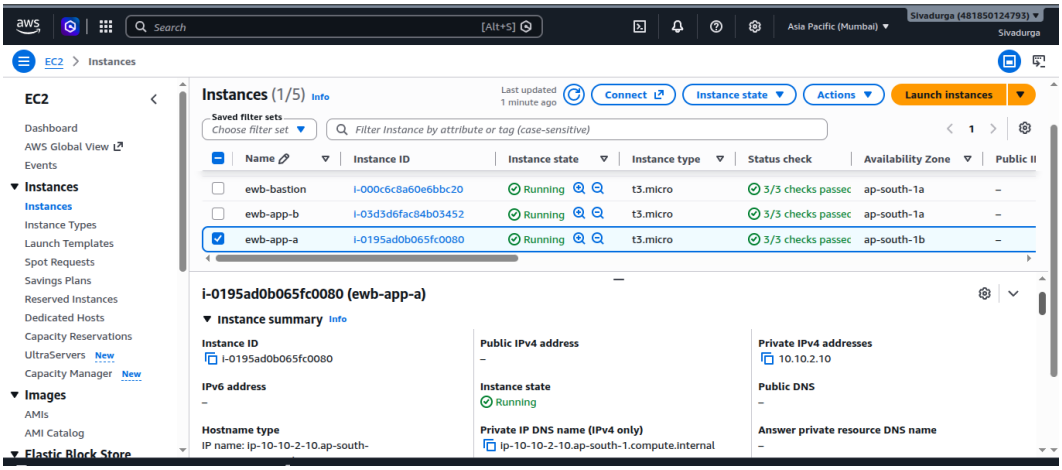
9. Evidence Gallery — EC2 Instances

Evidence E5 — Bastion instance



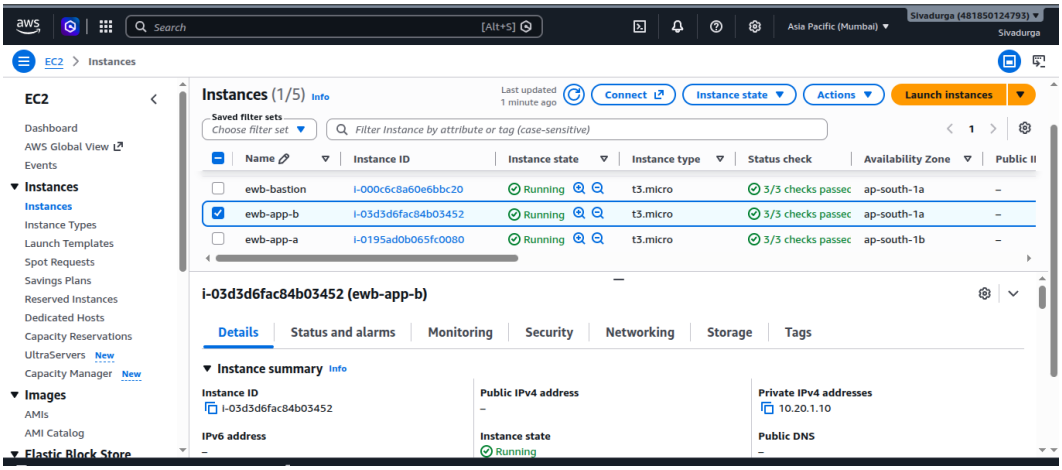
The selected instance is running. Its details show private IPv4 10.10.1.10 and public IPv4 13.127.110.57 at the time of capture.

Evidence E6 — App-A instance



The selected instance is running with private IPv4 10.10.2.10; no public IPv4 is displayed.

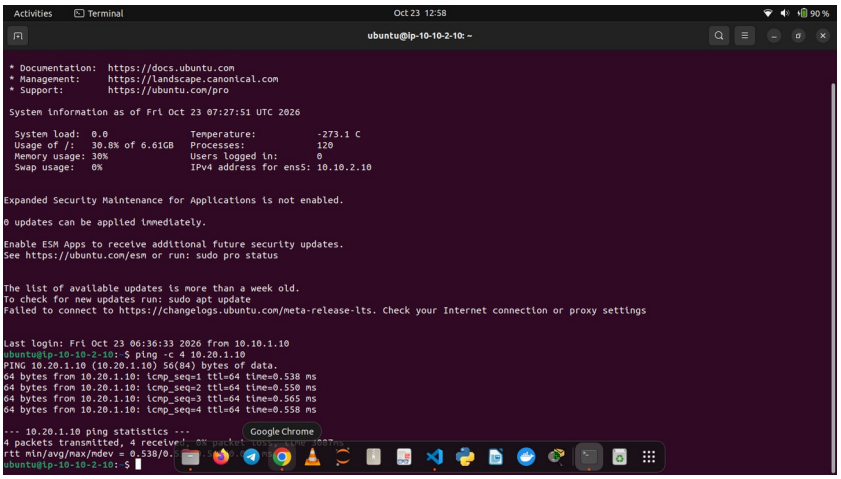
Evidence E7 — App-B instance



The selected instance is running with private IPv4 10.20.1.10; no public IPv4 is displayed.

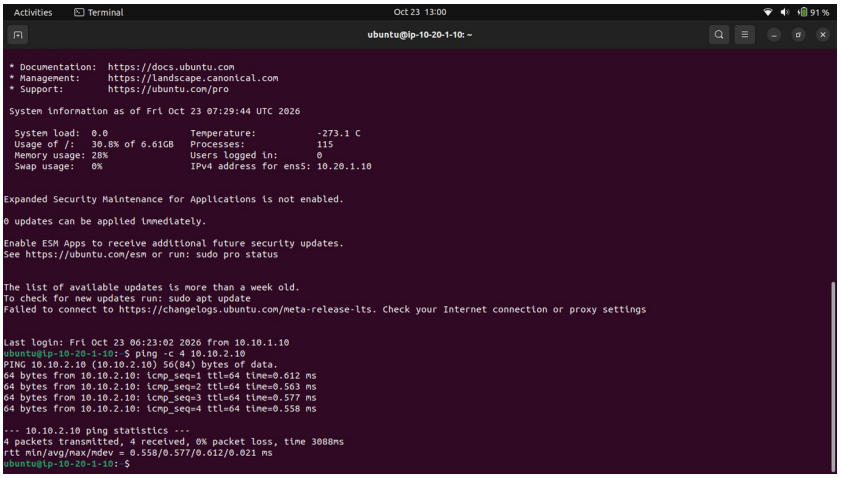
10. Evidence Gallery — Ping and SSH

Evidence E8 — App-A → App-B ping



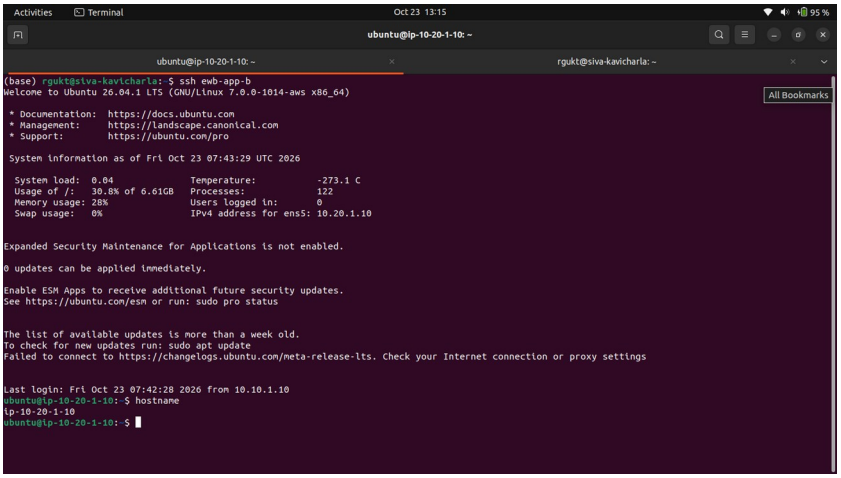
Command ping -c 4 10.20.1.10; 4 packets transmitted, 4 received, 0% packet loss.

Evidence E9 — App-B → App-A ping



Command ping -c 4 10.10.2.10; 4 packets transmitted, 4 received, 0% packet loss.

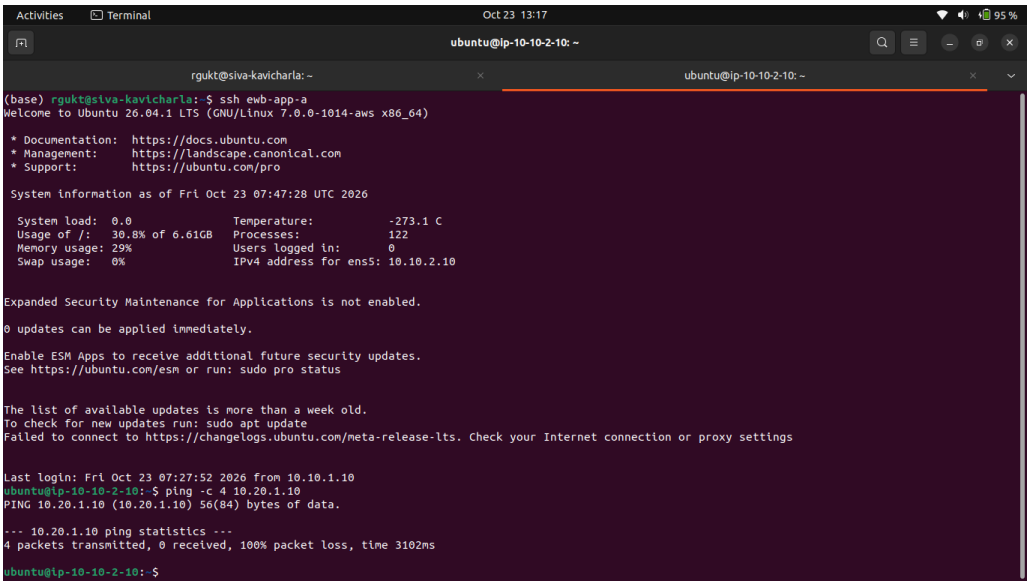
Evidence E10 — SSH session to App-B



The terminal shows SSH to ewb-app-b, Ubuntu login, and hostname ip-10-20-1-10. This screenshot supports access to App-B; the full ProxyJump command is not visible in the captured terminal.

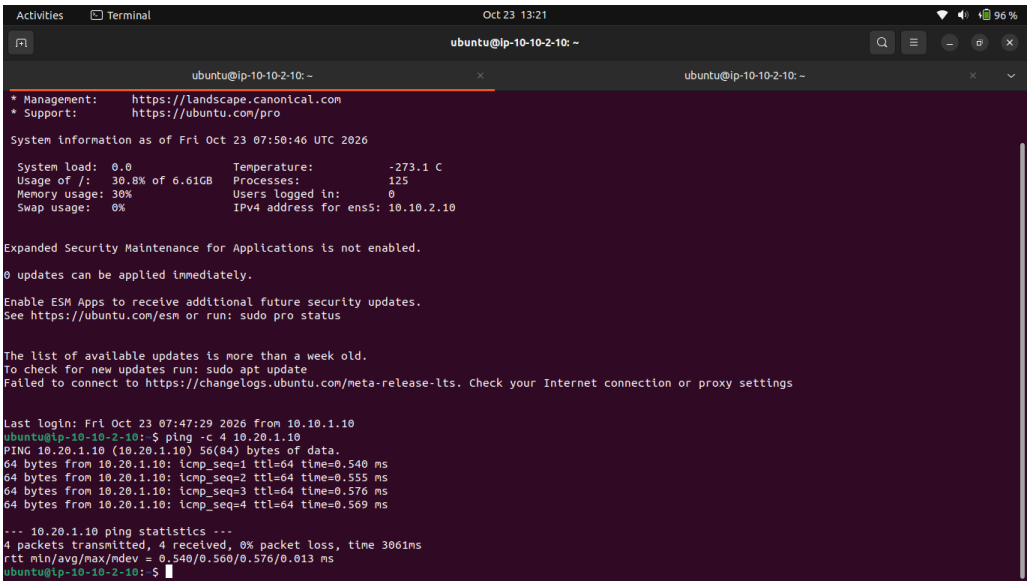
11. Evidence Gallery — Controlled Failure and Recovery

Evidence E11 — Controlled ICMP failure



App-A sends ping to 10.20.1.10 and receives no replies: 4 transmitted, 0 received, 100% packet loss.

Evidence E12 — ICMP recovery



After restoring the App-B ICMP rule, App-A receives four replies with 0% packet loss.

Evidence limitations: screenshots for security-group inbound rules, the two private route tables and subnet associations, and the initial baseline failure before peering/routes were not supplied. The initial baseline-failure screenshot was not captured. These gaps are disclosed rather than replaced with fabricated evidence.

12. Lessons Learned

? VPC peering requires non-overlapping VPC CIDR ranges and an active peering connection.

? Routes must exist on both sides because replies require a return path.

? The bastion subnet route table needs a route to VPC B when SSH ProxyJump connects to App-B.

? Security groups determine whether new SSH connections and ICMP Echo Requests are permitted.

? A negative test followed by a successful recovery test helps verify that a specific security rule controls connectivity.

? VPC peering is not transitive and does not provide internet access to the peer VPC.

13. Conclusion

The project demonstrates private communication between two VPCs through an active VPC peering connection. The supplied console screenshots confirm the peering status and VPC CIDRs. Terminal evidence confirms successful ping in both directions, a controlled ICMP block, recovery after the rule was restored, and an SSH session on App-B. The implementation used Ubuntu rather than the workbook's Amazon Linux 2023; this difference is documented. The initial baseline-failure screenshot was not captured.

Project summary

Item	Result
VPC peering	Active ? pcx-03fca277b8a48c690
Bidirectional ICMP	4 packets received; 0% loss in each direction
Controlled failure / recovery	Ping failed after ICMP rule removal and succeeded after restoration
SSH access	SSH session to App-B evidenced
OS deviation	Ubuntu used instead of Amazon Linux 2023